Week 2: functions, arrays, figures, calculus

Python syntax and code patterns. Keep it next to you while you work.

Writing a function

```
def sigmoid(x, slope=1.0, midpoint=0.0):
    """One line saying what it does."""
    return 1 / (1 + np.exp(-slope * (x - midpoint)))

sigmoid(0.5)                # defaults used
sigmoid(0.5, slope=4)       # say which one you mean
sigmoid(0.5, 4, 0.1)        # or give them in order

def two_things(x):
    return x.min(), x.max()

low, high = two_things(values)   # unpack what comes back
```

Your own module

```
from src.qmn_utils import sigmoid, gaussian

# after editing the .py file, reload it:
import importlib, src.qmn_utils
importlib.reload(src.qmn_utils)
```

Making arrays

```
np.array([1.0, 2.0, 3.0])
np.linspace(0, 6, 600)       # 600 points, both ends included
np.arange(0, 6, 0.01)        # step 0.01, end excluded
np.full_like(t, 0.05)        # same shape, one value
x.shape, x.size, len(x)
dt = t[1] - t[0]             # the spacing of a grid
```

Maths on a whole array

```
y = 2 * x + 1                # every element at once
np.exp(x), np.log(x), np.sqrt(x)
np.sin(x), np.cos(x)
np.abs(x), np.pi, np.e
x.mean(), x.max(), x.min(), x.sum()
np.allclose(a, b)            # equal to rounding error
```

Picking parts out

```
x[0], x[-1], x[10:20]
mask = x > 2                 # array of True and False
x[mask]                      # only where True
x[(x > 0) & (x < 1)]         # brackets, & not "and"
np.argmax(y), np.argmin(y)   # position of the extreme
t[np.argmax(y)]              # where the peak happens
np.abs(x)[error <= 0.02].max()
```

The shapes of the week

```
y = amp * np.exp(-t / tau)                  # decay
y = np.exp(-(x - mu)**2 / (2 * sigma**2))   # Gaussian
y = 1 / (1 + np.exp(-k * (x - x0)))         # sigmoid
y = amp * np.sin(2 * np.pi * f * t)         # sine
```

One figure

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(x, y, lw=2, ls="--", color="#d62728",
        label="tau = 1.5 s")
ax.axhline(0, color="gray", lw=0.8)
ax.axvline(x0, color="gray", lw=0.8)
ax.set_xlabel("time (s)")
ax.set_ylabel("amplitude")
ax.set_title("what this panel shows")
ax.set_xlim(0, 2); ax.set_ylim(-1, 1)
ax.legend(frameon=False, loc="lower left")
fig.tight_layout()
fig.savefig("figure.png", dpi=150, bbox_inches="tight")
plt.show()
```

Other kinds of panel

```
ax.scatter(x, y)                    one dot per observation
ax.hist(v, bins=20, alpha=0.7)      a distribution
ax.errorbar(x, y, yerr=e)           value with uncertainty
ax.text(x, y, "label")              writing on the axes
ax.semilogy(x, y)                   ax.plot with a log y axis
```

Several panels

```
fig, axes = plt.subplots(2, 1, figsize=(9, 5),
                         sharex=True)
axes[0].plot(t, signal); axes[0].set_ylabel("signal")
axes[1].plot(t, rate);   axes[1].set_xlabel("time (s)")
fig.suptitle("one title for both")
fig.tight_layout()
```

Pattern: one curve per parameter

```
fig, ax = plt.subplots()
for tau in [0.5, 1.5]:
    ax.plot(t, exp_decay(t, tau=tau), lw=2,
            label=f"tau = {tau} s")
ax.legend(frameon=False)
```

Checking yourself

```
assert abs(a - b) < 1e-12, f"differ at x={value}"
np.allclose(a, b)            # True if equal to rounding

print(f"tau=0.5 gives {f(0.5):.3f}, "
      f"tau=1.5 gives {f(1.5):.3f}")   # one f-string, two lines
```

Noise

```
rng = np.random.default_rng(7)      # 7 is the seed
noise = rng.normal(0, 0.15, size=t.size)
trace = clean + noise
i0 = np.argmin(np.abs(x))    # index of the value nearest zero
```

Finding a point, finding peaks

```
above = y >= 0.8             # array of True and False
if above.any():              # ask first: does it ever happen?
    first = np.argmax(above) # position of the first True
    x[first]                 # ... and where that is
# on an all False array argmax returns 0, silently

from scipy.signal import find_peaks
rate = np.gradient(y, x)
peaks, properties = find_peaks(rate, height=0.005)
x[peaks], rate[peaks]        # where the peaks are, how tall
# height ignores small bumps, distance= sets a minimum gap
```

Calculus, numerically

```
dy = np.gradient(y, x)       # slope at every point
d2y = np.gradient(dy, x)     # curvature

area = np.trapezoid(y, x)    # np.trapz on older NumPy
running = np.cumsum(y) * dt  # the accumulation
np.diff(y)                   # differences between neighbours
```

Pattern: a signal built from parts

```
steps = [(12, 0.15, 0.8),      # a list of tuples, one per part
         (28, 0.18, 0.5),
         (45, 0.07, 1.2)]

curve = np.full_like(sessions, 0.5)      # start from a baseline
for midpoint, height, sharpness in steps:
    curve = curve + transition(sessions, midpoint,
                               height, sharpness)
```